

Convertendo Tokens em Código

Mário Leite

Hoje em dia muitos programadores estão com medo de perder seus empregos face o avanço das Inteligências Artificiais (IA's) na codificação de programas computacionais, muito mais rápido que qualquer ser humano. Mas, é bom entender bem: eu disse CODIFICAÇÃO; o que é BEM DIFERENTE de PROGRAMAÇÃO. Entretanto, fica uma dúvida bem pertinente no ar: COMO AS IA's CODIFICAM UM PROGRAMA COMPUTACIONAL, planejado, algoritmizado e criado por um programador?!

O que é um Token?

Um **token** é a unidade básica de texto que um modelo de linguagem como eu processa; não é exatamente uma palavra, nem uma letra - é algo intermediário.

Como funciona na prática

O texto é dividido em pedaços chamados tokens antes de ser processado. Essa divisão segue padrões estatísticos da língua:

- Palavras comuns viram 1 token: **casa, the, amor**
- Palavras longas ou raras viram múltiplos tokens: incompreensível ==> incom + preen + sível
- Pontuação e espaços também são tokens: , . \n
- Números podem ser divididos: **12345 → 123 + 45**

Regra prática

Em inglês, 1 token ≈ 4 caracteres (ou ¾ de uma palavra); mas, em português, como as palavras tendem a ser mais longas, o consumo de tokens costuma ser um pouco maior.

Exemplos reais de tokenização:

Texto	Tokens
"Olá"	1
"Programação"	3–4
"1001 Programas em Python"	~6

Por que tokens importam?

Eles são a "moeda" dos modelos de linguagem:

- Janela de contexto: cada modelo tem um limite máximo de tokens que consegue processar de uma vez (entrada + saída juntas).
- Custo de API: cobranças são feitas por tokens consumidos (entrada e saída separados).
- Velocidade: mais tokens = mais tempo de processamento.

Quando você envia uma mensagem e eu respondo:

- O texto que você envia → tokens de entrada (input tokens)
- O texto que eu gero → tokens de saída (output tokens)

Ambos são contados e, na API da Anthropic, têm preços diferentes — os tokens de saída geralmente custam mais, pois exigem mais computação para serem gerados.

Um problema

Suponha que um professor precise calcular a média aritmética de 4 notas de um aluno e informar se ele está APROVADO (media $\geq 7,0$) ou REPROVADO (media $< 7,0$). Para um iniciante em programação, a dúvida é: como uma IA entende esse enunciado? A resposta está nos **tokens**: as peças mínimas com significado que o interpretador processa uma a uma.

A frase do prompt para a IA poderia ser esta:

"Crie um programa em Python que leia 4 notas de um aluno, calcule a média aritmética e informe APROVADO (media $\geq 7,0$) ou REPROVADO."

Para a IA, essa frase não é um bloco de texto qualquer; ela a processa **token a token**: fragmentos mínimos portadores de significado. São esses tokens do enunciado que ela vai traduzir nos tokens do código Python, por exemplo.

Como a IA lê o enunciado

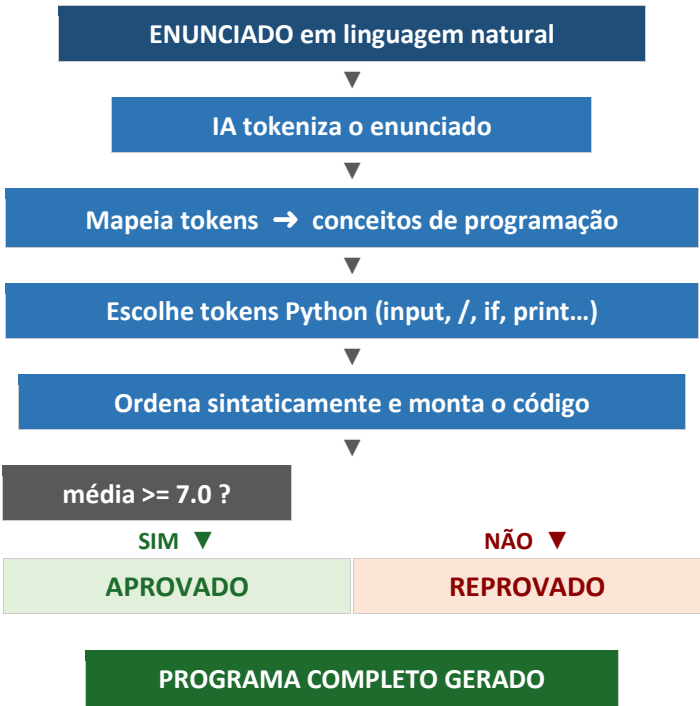
O modelo identifica as **unidades semânticas** do enunciado e as mapeia para conceitos de programação; veja a **tabela 1**.

Token do enunciado	→	Token Python gerado	Funcao no programa
"leia 4 notas"	→	float(input(...))	Entrada de dado numerico
"media aritmetica"	→	(n1+n2+n3+n4) / 4	Formula de calculo
"de um aluno"	→	nota1, nota2, nota3, nota4	4 variaveis nomeadas
"APROVADO / REPROVADO"	→	if / else + print()	Saida condicional
"media $\geq 7,0$ "	→	≥ 7.0	Operador relacional
"informe"	→	print(f"Media:{m:.2f}")	Exibicao formatada

Tabela 1 - Identificando o modelo semântico para a codificação

3 - O fluxo de geração de código

Internamente, a IA percorre este caminho para transformar o enunciado em código:



4 - Os tokens Python gerados

Ao final do processo, a IA produziu os seguintes tokens - cada um com papel preciso, como mostra a **tabela 2**.

#	Token Python	Categoria	Papel no programa
1	nota1 ... nota4	Identificadores	Guardam as 4 notas digitadas
2	=	Atribuicao	Liga o nome ao valor
3	float(input(...))	Funcoes aninhadas	Le e converte a nota para numero
4	(n1+n2+n3+n4) / 4	Expressao aritm.	Calcula a media
5	media	Identificador	Armazena o resultado
6	print(f"...{media:.2f}")	Saida formatada	Exibe a media com 2 decimais
7	if media >= 7.0:	Condicional	Testa o criterio de aprovacao
8	else:	Alternativa	Caminho para reprovacao
9	print("APROVADO")	Saida	Resultado positivo
10	print("REPROVADO")	Saida	Resultado negativo

Tabela 1 - Produzindo os tokens probabilisticamente

5 - O programa montado token a token

Encadeando os tokens na ordem sintática correta, o resultado e:

```
nota1 = float(input('Nota 1: '))
nota2 = float(input('Nota 2: '))
nota3 = float(input('Nota 3: '))
nota4 = float(input('Nota 4: '))

media = (nota1 + nota2 + nota3 + nota4) / 4

print(f'Media: {media:.2f}')

if media >= 7.0:
    print('APROVADO')
else:
    print('REPROVADO')
```

Cada linha e o resultado direto de um mapeamento: a **IA não inventou o código** - ela **traduziu** tokens do enunciado em tokens Python, na ordem e estrutura corretas. O que acontece “por baixo dos panos” é o seguinte: a cada passo, o modelo não “escolhe” um token; calcula uma **distribuição de probabilidade** sobre todos os tokens possíveis do vocabulário (que pode ter 50.000 a 100.000+ tokens, e então *amostra* dessa distribuição.

6 - Saida com notas reais

Com as notas 8,0 / 6,5 / 9,0 / 7,5:

Nota 1: 8.0
Nota 2: 6.5
Nota 3: 9.0
Nota 4: 7.5
Media: 7.75

APROVADO

Calculo: $(8,0 + 6,5 + 9,0 + 7,5) / 4 = 7,75 \Rightarrow \text{media} \geq 7,0 \Rightarrow \text{APROVADO}$

7 - Conclusão

O que chamamos de **geração de código** (codificação) **por IA** é, na essência, uma **tradução de tokens**: a IA recebe *tokens linguísticos* (palavras do enunciado), identifica seu significado no *domínio* da “programação” e emite *tokens sintáticos* (instruções, por exemplo, em Python) na ordem correta.

Compreender esse mecanismo ajuda a formular pedidos mais precisos - e a entender por que a qualidade do enunciado determina diretamente a qualidade do código gerado. Isto quer dizer que um prompt de solicitação é fundamental para a IA entenda perfeitamente o que se deseja, para produzir melhores respostas e informações ao usuário.